Janvier 2026

SIMULATEUR D'UNE MACHINE VIRTUELLE

Guide utilisateur



Yehor Chernyk & Tom Jougier

TABLE DES MATIERES

1 Installation 3

1.1 Prérequis 3

1.2 Compilation avec le Makefile 3

1.3 Compilation manuelle 3

1.4 Vérification de la compilation 3

2 Utilisation 3

2.1 Lancement du simulateur 3

2.2 Déroulement de l'exécution 4

2.3 Fichier intermédiaire 4

3 Format des fichiers assembleur 4

3.1 Structure générale 4

3.2 Étiquettes 4

3.3 Registres 5

3.4 Valeurs immédiates 5

4 Description des instructions 5

4.1 Instructions arithmétiques et logiques 5

4.2 Instructions de transfert de mémoire 5

4.3 Instructions de saut 6

4.4 Instructions d'entrée/sortie 6

4.5 Instructions diverses 6

5 Résolution des problèmes 7

5.1 Erreur : « Impossible d'ouvrir le fichier » 7

5.2 Erreur : « Erreur ligne X » 7

5.3 Le programme ne s'arrête pas 7

5.4 Division par zéro 7

5.5 Valeurs inattendues 8

5.6 Support 8



1 Installation

1.1 Prérequis

Compilateur C (GCC recommandé) et MinGW

1.2 Compilation avec le Makefile

Sur Linux et MacOS
Ouvrez un terminal dans le répertoire `src` du projet et exécutez :

```
make
```

Pour nettoyer les fichiers compilés :

```
make clean
```

Sur Windows
Avec MinGW installé, les mêmes commandes devraient fonctionner.
Si `make` n'est pas reconnu, exécutez `mingw32-make`.

1.3 Compilation manuelle

```
ggc -o assembleur.o -c assembleur.c -Wall
ggc -o processeur.o -c processeur.c -Wall
gcc -o main.o -c main.c -Wall
gcc -o simulateur assembleur.o processeur.o main.o
```

1.4 Vérification de la compilation

Après la compilation, vérifiez que l'exécutable `simulateur` a été créé.

2 Utilisation

2.1 Lancement du simulateur

Ouvrez un terminal dans le répertoire contenant l'exécutable `simulateur`.

Sur Linux et MacOS, exécutez `./simulateur` **`<fichier.txt>`**, avec fichier.txt le nom de votre fichier contenant le code assembleur (il doit être placé dans le même répertoire). Sur Windows, exécutez `simulateur` **`<fichier.txt>`**.

2.2 Déroulement de l'exécution

Assemblage : Le simulateur lit le fichier assembleur et le traduit en langage machine (fichier `hexa.txt`).

Vérification : Si des erreurs de syntaxe sont détectées, elles sont affichées avec le numéro de ligne.

Exécution : Le programme est chargé en mémoire et exécuté instruction par instruction.

Interaction : Si le programme utilise l'instruction `in`, vous devrez entrer des valeurs au clavier.

Résultats : Les instructions `out` affichent les résultats à l'écran.

2.3 Fichier intermédiaire

Lors de l'assemblage, un fichier `hexa.txt` est créé contenant le code machine en hexadécimal. Ce fichier peut être conservé pour analyse ou supprimé après exécution.

Notez que si vous exécutez deux programmes à la suite, le fichier hexadécimal du premier sera écrasé.

3 Format des fichiers assembleur

3.1 Structure générale

Chaque ligne ne peut contenir qu'une seule instruction, ou être vide.

Elle doit respecter le format suivant :

```
étiquette: instruction opérande1, opérande2, ...
```

Les espaces et tabulations sont autorisés pour l'indentation.

Notez que les commentaires ne sont pas encore autorisés.

3.2 Étiquettes

Les étiquettes permettent de marquer des positions dans le code pour les sauts :

```
debut: add r1, r0, #10
jmp debut
```

Leur nom ne doit être composé que de lettres, de chiffres et du caractère « _ » ; et doit obligatoirement être suivi du symbole deux points lors de la définition. Vous ne pouvez pas définir des étiquettes dont le nom est de la forme `rX`, avec `X ∈ {0, ..., 31}`.

Deux étiquettes déclarées avec le même nom provoqueront un message d'erreur lors de la compilation.

3.3 Registres

32 registres sont disponibles : r0 à r31. Ils sont tous initialisés à 0.

Notez que le registre r0 vaut toujours 0. Toute tentative d'écriture est ignorée.

3.4 Valeurs immédiates

Les valeurs immédiates sont toujours précédées d'un « # », sans exception. Elles peuvent être décimales (`#10`, `#-5`, `#0`) ou hexadécimales (`#h1A`, `#hFF`, `#h0A2B`). Dans le deuxième cas, elles débutent par le caractère « h ».

Les valeurs hexadécimales négatives sont en complément à deux. Par exemple, `#hFFFF` vaut `-1`.

4 Description des instructions

4.1 Instructions arithmétiques et logiques

Instruction	Description
or rd, rn, S	OU bit-à-bit : rd ← rn | S
and rd, rn, S	ET bit-à-bit : rd ← rn & S
xor rd, rn, S	XOR bit-à-bit : rd ← rn ^ S
add rd, rn, S	Addition : rd ← rn + S
sub rd, rn, S	Soustraction : rd ← rn - S
mul rd, rn, S	Multiplication : rd ← rn × S (16 × 16 bits)
div rd, rn, S	Division entière : rd ← rn / S

4.2 Instructions de transfert de mémoire

Instruction	Description
ldb rd, (rn)S	Charger 1 octet depuis (rn+S)
ldh rd, (rn)S	Charger 2 octets depuis (rn+S)
ldw rd, (rn)S	Charger 4 octets depuis (rn+S)
stb (rd)S, rn	Stocker 1 octet de rn vers (rd+S)

sth (rd)S, rn	Stocker 2 octets de rn vers (rd+S)
stw (rd)S, rn	Stocker 4 octets de rn vers (rd+S)

4.3 Instructions de saut

Instruction	Description
jmp S	Saut inconditionnel vers S
jzs S	Saut si Z=1 (résultat nul)
jzc S	Saut si Z=0 (résultat non nul)
jcs S	Saut si C=1 (retenue)
jcc S	Saut si C=0 (pas de retenue)
jns S	Saut si N=1 (résultat négatif)
jnc S	Saut si N=0 (résultat positif)

Les bits Z, N et C sont mis à jour après l'exécution d'une instruction, suivant le résultat de l'instruction.
Z est mis à 1 si le résultat est nul, 0 sinon.
N est la recopie du bit de poids fort du résultat (correspond au signe).
C est mis à 1 s'il y a une retenue. En cas de débordement, C est assigné au dernier bit perdu.
Les instructions de saut n'affectent pas ces 3 bits.

4.4 Instructions d'entrée/sortie

Instruction	Description
in rd	Lire un nombre depuis le clavier
out rd	Afficher le contenu de rd

L'instruction `in` n'accepte que des entiers en entrée. Toute autre entrée n'arrêtera pas le programme, mais renverra un résultat incohérent.

4.5 Instructions diverses

Instruction	Description
rnd rd, rn, S	Nombre aléatoire entre rn et S-1
hlt	Arrêt du programme

L'instruction `rnd` génère un nombre à partir de l'heure actuelle. Ainsi, si deux nombres sont générés consécutivement, ils auront la même valeur.

Si l'instruction `hlt` est absente, le programme s'arrête lorsqu'il rencontre la fin du fichier. Il est conseillé d'inclure cette instruction.

5 Résolution des problèmes

5.1 Erreur : « Impossible d'ouvrir le fichier »

Vérifiez que le nom du fichier est correct et qu'il est placé dans le répertoire courant.

Utilisez le chemin complet si le fichier est ailleurs.

Vérifiez les permissions en lecture du fichier.

5.2 Erreur : « Erreur ligne X »

Une erreur de syntaxe a été détectée à la ligne X.

Vérifiez :

- L'orthographe de l'instruction
- Le nombre d'opérandes (certaines instructions en ont 2, d'autres 3)
- La syntaxe des valeurs immédiates (`#10` pour décimal, `#hFF` pour hexadécimal)
- La présence de virgules entre les opérandes
- L'existence et le format des étiquettes
- Que les adresses de saut soient bien divisibles par 4

Plus généralement, référez-vous aux sections 3 et 4.

5.3 Le programme ne s'arrête pas

Causes possibles :

- Boucle infinie : vérifiez votre condition de sortie
- Instruction `hlt` manquante ou jamais atteinte
- Logique de saut incorrecte (`jzc` au lieu de `jzs` ou inversement)
- Manipulation de nombres négatifs dans un programme prévu pour des entiers positifs
- Erreur de conception de votre part (division par 0, adresse mémoire incorrecte, etc.)

Appuyez sur `Ctrl+C` pour interrompre le programme.

5.4 Division par zéro

Si vous effectuez une division par zéro, le simulateur met le flag C à 1 et place 0 dans le registre destination. Cependant, cela n'arrête pas le programme.

5.5 Valeurs inattendues

Si vous obtenez des valeurs différentes de celles attendues :

- Rappelez-vous que r0 vaut toujours 0
- La multiplication ne prend que les 16 bits de poids faible
- Les valeurs sont signées en complément à 2
- L'instruction `in` n'accepte que les entiers
- Vérifiez les overflows (le flag C indique un débordement)

5.6 Support

Pour plus d'aide, consultez :

- Le fichier *dev.pdf* pour comprendre le fonctionnement interne
- Les fichiers de test dans le dossier `resources` pour des exemples supplémentaires